WPT Day 7 Evening

Tue 27 Oct 26 · Why the raw `http` module does not scale for real apps, Express.js as a Node backend framework, method-based routing, testing with Postman, response body/status/headers, URL parameters (`/sum` , `/factorial`), and I/O-intensive vs CPU-intensive technology choice

---

Table of Contents

1. Limitations of the Raw HTTP Module
2. Why Express.js (and How to Set It Up)
3. Routing in Express
4. Postman and Response Anatomy
5. URL Parameters: Sum and Factorial
6. I/O-Intensive vs CPU-Intensive Workloads
7. Combined Worked Example: Express Calculator API
8. Quick Revision Checklist
9. Practice Exercises
10. Practice Answers

Limitations of the Raw HTTP Module

The problem one callback cannot solve

In the previous sessions you built a local server with Node's built-in `http` module: one `createServer` callback, one `res.end( … )` , done. That works for a hello-world demo. It collapses the moment the application must behave differently for different URLs and methods — which is exactly what every real product needs.

Mechanism: one function for every request

With raw `http.createServer` , **a single callback handles every incoming request**, regardless of:

- **URL** — base `/` , `/login` , `/signup` , `/profile` , `/search` , …
- **Method** — GET, POST, PUT, DELETE on the same URL

Whatever the client hits, the **same function runs**. Without extra manual work, every URL returns the **same response**. There is no built-in "this path means login, that path means search."

To differentiate, you must write `if` / `else` **(or** `switch` **) on** `req.url` **inside that one callback**. Scale that up:

What you need	What raw HTTP forces you to write
100 different URLs	~100 nested / chained conditions in one function
GET vs POST vs PUT vs DELETE on the same URL	Still more nesting inside each URL branch
Readable, debuggable, maintainable code	The opposite — one giant callback

Why real apps need per-URL, per-method processing

E-commerce, social media, and trip-booking apps do not return one string for every hit. The lecture's concrete framing:

- Base URL, `/login`, `/signup`, `/search`, `/profile` each expect **different** responses and processing.
- **MakeMyTrip** example: `/hotels` and `/railways` return completely different content — hotel search vs railway booking. One shared callback with nested conditions is the wrong abstraction.

Exam-ready takeaway

- Raw HTTP's root pain: **one callback, every request** → differentiation only via manual `req.url` / method conditions.
- Scaling that pattern becomes **unmanageable** — hard to read, debug, and maintain.
- Lecture's diagnosis: **writing everything from scratch** is the root problem; a **library or framework** exists to solve it.

> **Exam:** State the limitation as "single `createServer` callback for all URLs/methods; multi-URL apps need nested `req.url` conditions." Mention a real product contrast (e.g. MakeMyTrip `/hotels` vs `/railways`).

> **Pitfall:** Thinking "I can just keep adding `if (req.url === …)` forever." Technically yes; professionally that is what Express routing replaces.

Why Express.js (and How to Set It Up)

The problem Express answers

You already know *how* to open a port with `http`. You need a **structured way** to map URLs and methods to handlers, with a scalable project shape, without reinventing routing, response helpers, and common security-related defaults for every app.

What Express is

Express is a **small but complete framework** for backend / server development in Node.js.

- Installed as a module: `npm i express` (`i` = shorthand for `install`).
- Gives you: predefined functions, a scalable code structure, inbuilt security-oriented features, and **routing support out of the box**.
- You no longer need to think about low-level HTTP handling from scratch for everyday APIs.

Other Node backend frameworks also exist (lecture example: **Nest.js**). On the frontend side, the lecture pairs **React.js** (library) for UI with **Express** for the backend.

Library vs framework (lecture distinction)

	Library	**Framework**
Size / scope	Small, reusable piece of code with predefined functions	Full(er) ecosystem: structure, end-to-end inbuilt support
Lecture example	**React**	**Angular**, **Express**
Routing	React has **no** built-in routing; needs third-party packages (e.g. React Router, Axios for HTTP)	Angular includes routing, HTTP, animations, responsive UI support built in; Express gives structure, routing, request/response handling, and security without hunting elsewhere for basics

Memorable framing: a **library** is a toolbox you call; a **framework** is an ecosystem that already knows how a typical app is shaped.

Dependencies vs dev dependencies

Kind	**When needed**	**How marked**	**Examples from lecture**
dependencies	Production runtime	Default `npm i <pkg>`	Express

Kind	When needed	How marked	Examples from lecture
devDependencies	Development / build / test only — **not** needed in production	`npm i <pkg> --save-dev`	TypeScript (compiles to JS before deploy), unit-testing tools

Production packages like Express stay under `dependencies` in `package.json`. Dev-only tools are marked with `--save-dev` so production environments can omit them.

Setting up an Express project (lecture steps)

1. **Create a dedicated folder** (e.g. `my-server`) and open it in VS Code.
2. **Initialize `package.json` before writing any code:**
   - `npm init` — prompts for name, version, description, entry point, author, …
   - `npm init -y` — skips prompts; accepts defaults (folder name as package name, version `1.0.0`, `index.js` as main).
3. **Install Express:** `npm i express`
   - Generates / updates `package-lock.json` and `node_modules`.
   - Express appears under `dependencies` in `package.json`.
   - All Express library files live inside `node_modules`.
4. **Create `index.js`** as the application entry point.
5. **Optional:** set `"type": "module"` in `package.json` to use ES `import` syntax (as in the Day 7 morning arc).

Creating and starting the server (three steps)

Step 1 — import Express

```
import Express from 'express';
// CommonJS equivalent: const Express = require('express');
```

- No curly braces — imports the **entire** module (default export style).
- The binding name is yours: `Express`, `ABC`, `Library`, etc. — the name is local; the package is still `'express'`.

Step 2 — create the application object

```
const app = Express();
```

- Calling `Express()` creates and returns the **server / application object**.

- `app` **is** your server object — you do **not** need the raw `http` module or `createServer` anymore for this pattern.

Step 3 — start listening

```
app.listen(6500);
```

- Without `app.listen( … )`, the application **never starts**.
- Base URL for the lecture's server: **`127.0.0.1:6500`**.

Background: `127.0.0.1` is the loopback address — "this machine." Port `6500` is just the number the process binds to; the browser/Postman must use the same host and port.

> **Exam:** Express = Node backend framework; install with `npm i express`; three steps = import → `const app = Express()` → `app.listen(port)`.

> **Pitfall:** Forgetting `app.listen` and wondering why nothing answers on `127.0.0.1:6500`. The process may run, but nothing is listening.

Routing in Express

The problem routing solves

You need a clean answer to: *which URL + which HTTP method runs which logic and returns which response?* Raw HTTP made you invent that map with nested conditions. Express makes the map a first-class API.

Definition

Routing = mapping a **URL** and a **request method** to specific processing logic and a response. It decides which URL gets which response and how the request is processed.

One predefined function per HTTP method

Method	Express function
GET	`app.get`
POST	`app.post`

Method	Express function
PUT	`app.put`
DELETE	`app.delete`

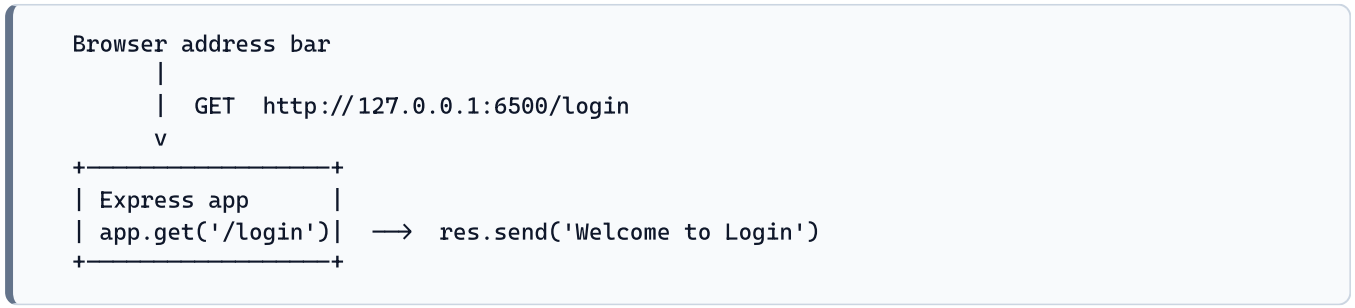
Each call takes **two arguments**:

1. **URL path** — only the part after the host:port slash (e.g. `/`, `/login`, `/sum`).
   - You do **not** repeat `127.0.0.1:6500` in every route; the server already knows IP/port from `app.listen`.
   - Base URL is represented as `/` (just a slash).
   - **Always include the leading slash.** Omitting it is bad practice and is flagged by code-quality tools like **SonarQube**.
2. **Callback** `(req, res) ⇒ { … }` — runs when a matching request arrives.

Lecture route examples

```
app.get('/', (req, res) ⇒ res.send('Hello World'));
app.get('/login', (req, res) ⇒ res.send('Welcome to Login'));
// app.get('/sum', …) – later replaced with URL parameters
```

- Multiple routes coexist; each URL is processed **independently**.
- Typing a URL in the browser sends **GET by default** — you do not choose the method in the address bar.



Exam: Routing = URL + method → handler. Know `app.get` / `post` / `put` / `delete`, path as `/…`, callback `(req, res)`.

Pitfall: Writing paths without a leading `/`, or stuffing the full `http://127.0.0.1:6500/login` string into `app.get`.

Postman and Response Anatomy

Why the browser is not enough for API testing

The browser is fine for a quick GET check, but it shows **only the response body**. You get no clear view of:

- Status code
- Time taken
- Response size

Postman (desktop app, or VS Code **Postman** / **Thunder Client** extensions) shows the full technical picture:

What you see	Example from lecture
Response body	Actual data returned
Status code	e.g. `200 OK`
Time taken	e.g. ~12 ms for Hello World, ~15 ms for `/sum`
Size	e.g. ~238 bytes Hello World, ~235 bytes `/sum`

Postman also lets you **choose the method** from a dropdown: GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS.

Practical notes from the session:

- Chrome-app Postman is **no longer supported** — use the desktop app or a VS Code extension.
- Thunder Client works the same idea inside VS Code.
- Send with `http://` (not `https://`) to `127.0.0.1:6500/[path]`, then click Send.

Every response has three parts

Part	Meaning
Body	Actual data sent (string or JSON)
Status	HTTP status code — success or failure signal
Headers	Metadata about the response

There is no such thing as a “response request.” A response is body + status + headers only.

Response body types

What you send	Example	Typical `content-type`
String	`res.send('Hello World')`	`text/html`
JSON object	`res.send({ message: 'Hello World' })`	`application/json`

Lecture rule of thumb: **real-time / production-style servers should always send JSON**, not plain strings.

Status codes

- Set explicitly: `res.status(200).send( ... )` or `res.status(400).send( ... )`.
- **Default is 200** if you do not set a status.
- Codes are **predefined** (roughly 100 through the 500-series). **Custom status codes do not exist.**

Code	Meaning in lecture
200	OK — success
400	Bad Request — invalid client input
500-series	Server errors

Response headers (Postman Headers tab)

Header	Role
`x-powered-by`	Identifies the framework (e.g. Express)
`content-type`	`application/json` or `text/html` depending on what was sent
`content-length`	Size of the response body in bytes
`date`, `connection`, `etag`	Additional metadata

Metadata = data about data (size, type, sender, timestamp, ...). Headers are readable programmatically in a client application if needed.

> **Exam:** Response = body + status + headers. JSON preferred for APIs. 200 default; 400 for bad client input; status codes are not free-form.

> **Pitfall:** Judging an API only from the browser's visible text. Always verify status and headers in Postman (or Thunder Client) when debugging.

URL Parameters: Sum and Factorial

The problem: getting non-sensitive data into the handler

APIs often need input from the client: numbers to add, an id to look up, cities to search. One common channel is the **URL itself** — **URL parameters** (path parameters in Express).

What belongs on the URL (and what must never)

Suitable on URL	Never on URL
Source / destination city, search query, item type	Passwords, OTPs, UPI PINs
MakeMyTrip-style flight search fields: DEL, IXL, travel date, trip type, cabin class	Anything secret or credential-like

Sensitive values must travel by safer channels (later: body, headers, secure sessions) — **never** as visible path segments.

Declaring parameters: colon syntax

```
app.get('/sum/:a/:b', (req, res) => { /* … */ });
app.get('/factorial/:num', (req, res) => { /* … */ });
```

- `/sum/:a/:b` declares two parameters named `a` and `b`.
- `/factorial/:num` declares one parameter named `num`.
- Names can be anything (`a` / `b` , `n1` / `n2` , `x` / `y`); the **colon** tells Express "this segment is a parameter."
- Client sends values in the path: `/sum/3/5` , `/factorial/5` .

Reading parameters: `req.params`

Express builds a plain object from all URL parameters:

Client URL	`req.params`
`/sum/3/5`	`{ a: '3', b: '5' }`
`/factorial/5`	`{ num: '5' }`

Read with `req.params.a`, `req.params.b`, `req.params.num`.

Critical: values are **always strings**. Arithmetic without conversion fails the way JavaScript always fails string math:

Expression	Result
`'3' + '5'` (no `parseInt`)	`'35'` — concatenation
`parseInt(req.params.a) + parseInt(req.params.b)`	`8` — numeric addition
`parseInt('abc')` etc.	`NaN`; `NaN + NaN` → `NaN`

Sum route (lecture)

```
app.get('/sum/:a/:b', (req, res) => {
  const z = parseInt(req.params.a) + parseInt(req.params.b);
  res.send({ sum: z }); // JSON, status 200 by default
});
```

Test: `/sum/7/5` → `{ sum: 12 }`.

Factorial route (lecture)

```
app.get('/factorial/:num', (req, res) => {
  const n = parseInt(req.params.num);
  if (n < 0) {
    return res.status(400).send({ message: 'Negative values are not allowed' });
  }
  let fact = 1;
  for (let i = 1; i ≤ n; i++) {
    fact *= i;
  }
  res.status(200).send({ factorial: fact });
});
```

Input	Behaviour
Valid (e.g. `5`)	`200` + `{ factorial: 120 }`

Input	Behaviour
Negative	`400 Bad Request` + `{ message: 'Negative values are not allowed' }`
Symbols / non-numeric	Validate with a **regular expression** to allow only numeric input (lecture call-out)

Restart after every change

After editing source, **restart the server** (`Ctrl+C` , then `node index.js`). Node does not hot-reload your routes by default in this workflow.

> **Exam:** Path params use `:name` ; read via `req.params` (strings); `parseInt` before arithmetic; 400 for invalid client data (e.g. negative factorial).

> **Pitfall:** Forgetting `parseInt` and celebrating `{ sum: '35' }` as "correct." That is concatenation, not addition.

I/O-Intensive vs CPU-Intensive Workloads

The problem: picking the right backend technology

Technology choice is not fashion — it depends on **what the application mostly does**.

Two workload kinds

Workload	What it means	Lecture examples	Preferred stack (lecture)
I/O-intensive	Heavy input/output: files, database calls, network requests	Typical web APIs waiting on DB / network	**Node.js + Express** — non-blocking, event-driven
CPU-intensive	Heavy computation: large-scale analysis, big-data pipelines	Number-crunching, processing-heavy jobs	**Java (e.g. Spring Boot)** — true multi-threading

Why Node fits I/O and struggles with CPU

- Node.js / JavaScript shine at **I/O-intensive** backends because of the **non-blocking, event-driven** architecture — while one request waits on the database, the event loop can serve others.

- For **CPU-intensive** work, Node struggles: JavaScript is **single-threaded** with **no true multi-threading** in the lecture's framing.
- A workaround exists (**worker threads**), but the lecture stresses it is a **workaround**, not a native multi-threaded design.
- **Java / Spring Boot** provides true multi-threading and is preferred when the workload is CPU-bound.

Choosing between **Node.js/Express** and **Java/Spring Boot** comes down to this distinction.

> **Exam:** I/O-heavy → Node/Express; CPU-heavy → Java/Spring Boot. Mention single-threaded JS vs true multi-threading.

> **Pitfall:** Claiming "Node can do everything equally well" without naming the I/O vs CPU trade-off the lecture insists on.

Combined Worked Example: Express Calculator API

One copy-paste-runnable project that ties together **project init**, **Express install**, **ES modules**, **method routing**, **JSON responses with status codes**, and **URL parameters** for sum and factorial — the practical arc of the evening.

Step trace

1. Create folder `express-calc`, open in VS Code.
2. `npm init -y` then set `"type": "module"` in `package.json`.
3. `npm i express` — Express under `dependencies`; `node_modules` + lockfile appear.
4. Write `index.js` as below.
5. `node index.js` → listen on `6500`.
6. Test in Postman with `http://127.0.0.1:6500/...` (GET).

`package.json` (illustrative)

```
{
  "name": "express-calc",
  "version": "1.0.0",
  "description": "WPT Day 7 Evening combined example",
  "main": "index.js",
  "type": "module",
  "scripts": {
```

```
    "start": "node index.js"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.21.0"
  }
}
```

`index.js` (complete)

```
import Express from 'express';

const app = Express();
const PORT = 6500;

// Base URL — string body (demo); APIs below use JSON
app.get('/', (req, res) => {
  res.send('Hello World');
});

app.get('/login', (req, res) => {
  res.send('Welcome to Login');
});

// /sum/7/5 → { "sum": 12 }
app.get('/sum/:a/:b', (req, res) => {
  const a = parseInt(req.params.a, 10);
  const b = parseInt(req.params.b, 10);
  const z = a + b;
  res.status(200).send({ sum: z });
});

// /factorial/5 → { "factorial": 120 }
// /factorial/-1 → 400 + message
app.get('/factorial/:num', (req, res) => {
  const raw = req.params.num;

  // Allow digits only (optional leading +); reject symbols / letters
  if (!/^\d+$/.test(raw)) {
    return res.status(400).send({ message: 'Only non-negative numeric input is
allowed' });
  }

  const n = parseInt(raw, 10);
  if (n < 0) {
    return res.status(400).send({ message: 'Negative values are not allowed' });
  }
```

```
  let fact = 1;
  for (let i = 1; i ≤ n; i++) {
    fact *= i;
  }
  res.status(200).send({ factorial: fact });
});

app.listen(PORT, () ⇒ {
  console.log(`Listening on http://127.0.0.1:${PORT}`);
});
```

End-to-end behaviour

Request	Expected
GET `/`	Body `Hello World` ; `content-type` tends toward `text/html` ; status 200
GET `/login`	`Welcome to Login`
GET `/sum/7/5`	`{ "sum": 12 }` JSON; status 200
GET `/sum/3/5` without `parseInt` (if you remove it)	Would become string `'35'` — the lecture's trap
GET `/factorial/5`	`{ "factorial": 120 }`
GET `/factorial/-1` or non-numeric	`400` + error message object
Edit code then refresh without restart	Old behaviour until `Ctrl+C` + `node index.js`

```
Postman  GET http://127.0.0.1:6500/sum/7/5
               |
               v
        app.get('/sum/:a/:b')
               |
               |  req.params = { a: '7', b: '5' }
               |  parseInt → 7 + 5 = 12
               v
        res.status(200).send({ sum: 12 })
               |
               +──→ body: {"sum":12}
               +──→ status: 200
               +──→ headers: content-type application/json, ...
```

Quick Revision Checklist

Topic	One-line takeaway
Raw HTTP limits	One `createServer` callback for all URLs/methods → nested `req.url` hell at scale.
Why Express	Small complete **Node backend framework**: structure, routing, security helpers out of the box.
Library vs framework	Library = reusable functions (React); framework = fuller ecosystem (Angular, Express).
Setup	Folder → `npm init` / `-y` → `npm i express` → `index.js` → optional `"type": "module"`.
Start server	`import Express` → `const app = Express()` → `app.listen(6500)`.
Routing	`app.get` / `post` / `put` / `delete` (path, handler); path is `/…` only.
Browser vs Postman	Browser ≈ body only; Postman shows body, status, time, size, method choice.
Response parts	**Body + status + headers** only; prefer JSON for real APIs.
Status codes	Default 200; 400 bad client input; codes are predefined, not custom.
URL params	`:name` in path; `req.params` (strings); `parseInt` before math; never put secrets in URL.
Restart	Change source → stop with Ctrl+C → `node index.js` again.
I/O vs CPU	I/O-heavy → Node/Express; CPU-heavy → Java/Spring Boot (true multi-threading).

Practice Exercises

Concept

1. Why does a single `http.createServer` callback become unmanageable for an app like MakeMyTrip with many distinct URLs?
2. In the lecture's terms, how does a **library** differ from a **framework**? Place React, Angular, and Express in that distinction.
3. What three parts does every HTTP **response** have? Why is "response request" not a valid term?

4. Why must URL parameter values be passed through `parseInt` (or similar) before addition in a `/sum/:a/:b` route?
5. When would you choose Node.js/Express over Java/Spring Boot, and when the reverse, according to the lecture?

Application

6. Write the exact sequence of npm/shell steps and the three Express lines needed to get a server listening on port `6500` with ES module syntax (assume empty folder).
7. Write an `app.get` route for `/profile` that sends the JSON `{ "message": "Welcome to Profile" }` with an explicit status `200`.
8. Write the factorial route logic that returns status `400` with `{ "message": "Negative values are not allowed" }` when `num` is negative, and `{ "factorial": … }` with status `200` otherwise (assume `num` is already a number).

Predict output / behaviour

9. Route is `app.get('/sum/:a/:b', …)` with `res.send({ sum: req.params.a + req.params.b })` (no `parseInt`). Client requests `/sum/3/5`. What is the JSON body?
10. You change `index.js` while `node index.js` is still running, save the file, and immediately hit `/login` in Postman. Do you see the new string? Why or why not?
11. `res.send({ message: 'Hello World' })` with no `res.status( … )`. What status does Postman typically show, and what `content-type` family do you expect compared to `res.send('Hello World')`?

Spot the bug

12. A student writes `app.get('login', (req, res) ⇒ res.send('Welcome to Login'))` and cannot hit the route at `http://127.0.0.1:6500/login`. What is wrong?
13. A student puts a user's OTP in the path as `/verify/:otp` "because URL parameters are convenient." What lecture rule does this violate, and what should they do instead at a high level?
14. Find the mistake: "I'll invent status code 999 so my API can mean 'almost success'."

Practice Answers

1.

Because raw HTTP runs **one callback for every request**. Differentiating `/hotels`, `/railways`, login, search, etc. means stacking `if` / `else` on `req.url` (and nesting again for methods). At

product scale that is unreadable and hard to maintain — the lecture's motivation for Express routing.

2.
A **library** is a small reusable set of predefined functions; a **framework** is a fuller ecosystem with structure and end-to-end built-in support. **React** = library (no built-in routing; needs third-party helpers). **Angular** and **Express** = framework side of the distinction (Express for Node backend structure/routing/security; Angular for full frontend stack features).

3.
Body, **status**, and **headers**. "Response request" is not a real term — those three parts fully describe a response.

4.
`req.params` values are **always strings**. Without conversion, `'3' + '5'` is concatenation → `'35'`, not arithmetic `8`.

5.
I/O-intensive apps (files, DB, network) → Node.js/Express (non-blocking, event-driven). **CPU-intensive** apps (heavy processing, big-data style pipelines) → Java/Spring Boot (true multi-threading). Worker threads in Node are a workaround, not the native story.

6.

```
npm init -y
npm i express
```

Then in `package.json` set `"type": "module"`, and in `index.js`:

```
import Express from 'express';
const app = Express();
app.listen(6500);
```

7.

```
app.get('/profile', (req, res) => {
  res.status(200).send({ message: 'Welcome to Profile' });
});
```

8.

```
if (num < 0) {
  return res.status(400).send({ message: 'Negative values are not allowed' });
}
```

```
let fact = 1;
for (let i = 1; i ≤ num; i++) {
  fact *= i;
}
res.status(200).send({ factorial: fact });
```

9.

`{ "sum": "35" }` — string concatenation of `'3'` and `'5'`, not `8`.

10.

No — not until you **restart** the process (`Ctrl+C`, then `node index.js`). The running process keeps the old loaded code.

11.

Status defaults to **200**. JSON object → `content-type` around `application/json`; plain string → typically `text/html`.

12.

Missing leading `/` — path must be `'/login'`, not `'login'`. SonarQube-style tools flag omitted slashes; Express will not match the browser's `/login` the way the student expects.

13.

Passwords, OTPs, UPI PINs must **never** appear in the URL. URL params are for non-sensitive data (cities, search terms, ids of that kind). Use a safer channel for secrets (e.g. request body over HTTPS in later patterns) — do not put OTP in the path.

14.

HTTP status codes are **predefined** (100 through 500-series in the lecture's framing). You cannot invent custom codes like 999; clients and intermediaries will not understand them as standard signals.